

APEX System Pipeline

Audience: Researchers tracing **actual** execution order in `backend/src`.

All HTTP routes below assume the app's `/v1` prefix (`app.py`).

End-to-End Flow (Trainee)

1. **Create session** — `POST /sessions` → `SessionService.create_session` (may return existing active session for same user+case unless `force_new`). Freezes `evaluator_plugin`, `patient_model_plugin`, `metrics_plugins` on `Session` (`domain/entities/session.py`).
2. **Submit turn (text)** — `POST /sessions/{session_id}/turns` → builds `DialogueService(db, OpenAIAdapter(), SimpleRuleNLU(), ...)` → `process_user_turn`.
3. **Submit turn (audio)** — `POST /sessions/{session_id}/audio` → `WhisperAdapter.transcribe_audio` + `AudioToneAdapter.analyze_audio` → same `process_user_turn` with `TurnCreate(text=transcript, voice_tone=...)`.
4. **Close** — `POST /sessions/{session_id}:close` → `SessionService.close_session` (sets `state="completed"`, `ended_at`, `duration_seconds`) → `ScoringService.generate_feedback(session_id)` → returns `FeedbackResponse`.
5. **Fetch feedback later** — `GET /sessions/{session_id}/feedback` reads persisted `Feedback` via `FeedbackRepository`.

WebSocket (optional): `WebSocket /ws/sessions/{session_id}` under `/v1` (`ws_controller.websocket_dialogue`) — query param `token` (Supabase JWT) — same `DialogueService.process_user_turn` as HTTP.

Single Turn: `DialogueService.process_user_turn`

Order in `services/dialogue_service.py`:

1. Load **session** and **case**; compute next `turn_number`.
2. Build `DialogueState(session)`.
3. **analyze_user_input** (`services/turn_analysis.py`) using `NLUPipeline.analyze` (`services/nlu_pipeline.py`) with the injected `SimpleRuleNLU` for spans, questions, tone, EO signals.
4. `StageTracker.detect_stage(turn_data.text, session)` then `update_session_stage` — persists `current_spikes_stage` (`StageTracker.STAGES`: `setting`, `perception`, `invitation`, `knowledge`, `emotion`, `strategy`).
5. Insert **user** `Turn` (`role="user"`) with `metrics_json`, `spans_json`, `spikes_stage`.
6. Load `conversation_history` from prior turns; set `state.case`, `state.session`, `state.conversation_history`.
7. `_instantiate_patient_model(session)` — `PluginRegistry.get_patient_model(plugin_key)` or `_load_class_from_path` + `register`; `plugin_cls()` (new instance each turn).
8. `await patient_model.generate_response(state, turn_data.text)`.
9. **analyze_assistant_response** on patient text (EO detection for assistant side, latency).

10. If `enable_tts` and adapters present: synthesize audio, store via `storage_adapter`, set `audio_expires_at`.
11. Insert **assistant** Turn (role="assistant"); return `TurnResponse.model_validate(created_turn)`.

Note: `sessions_controller` passes `OpenAIAdapter` into `DialogueService`, but `DefaultLLMPatientModel` builds its own `OpenAIAdapter` or `GeminiAdapter` from `get_settings().default_llm_provider`. The constructor `llm_adapter` on `DialogueService` is not used on this path for patient text generation.

Scoring Trigger

Event	Code path
Trainee/API close	<code>sessions_controller.close_session_and_get_feedback</code> → <code>close_session</code> then <code>ScoringService.generate_feedback(session_id)</code> .

There is no automatic scoring on the last turn without calling `:close`.

ScoringService.generate_feedback (Evaluator + Metrics)

`services/scoring_service.py`:

1. Load session; resolve `evaluator_plugin` from session row, else `settings.evaluator_plugin`.
2. `PluginRegistry.get_evaluator(plugin_key)` or `_load_class_from_path + register_evaluator`.
3. `evaluator = plugin_cls(); feedback = await evaluator.evaluate(self.db, session_id)`.
4. `_run_and_store_metrics_plugins(session_id)` — parses `session.metrics_plugins` JSON list; for each id, `MetricsPlugin.compute(db, session_id)`; writes `session.metrics_json = json.dumps({ plugin_id: result, ... })`; `session_repo.update`.
5. Return `feedback`.

Bundled evaluators (see `plugins/evaluators/`):

- `ApexBaselineEvaluator` → `generate_feedback_rule_only`
- `ApexHybridEvaluator` → `generate_feedback_hybrid`
- `ApexHybridV2Evaluator` → `generate_feedback_hybrid_v2`

Metrics plugins run **after** `evaluate` returns, still inside `generate_feedback`.

Research Export (Admin Only)

`controllers/research_controller.py` → **ResearchService** (`services/research_service.py`):

- `GET /research/sessions` — anonymized session list.
- `GET /research/sessions/{anon_session_id}` — one session payload.
- `GET /research/export` — JSON download `research_export.json`.
- `GET /research/export.csv` — flattened CSV.
- `GET /research/export/metrics.csv` — per-session metrics stream.

- GET /research/export/transcripts.csv — all transcripts.
- GET /research/export/session/{anon_session_id}.csv — one transcript CSV.

Anon id: `generate_anon_session_id = anon_ + SHA-256(session_id + research_anon_salt)[:12]`.

Numbered Pipeline Summary

1. User authenticated (Supabase JWT — see [core/deps](#), [core/security](#)).
2. Session created or reused; plugin ids frozen on row.
3. Each turn: NLU (user) → SPIKES stage → persist user turn → PatientModel → NLU (assistant) → persist assistant turn.
4. Close: session marked **completed**.
5. Evaluator plugin runs → feedback persisted (via evaluator/scoring helpers).
6. Metrics plugins run → **sessions.metrics_json** updated.
7. Research exports read DB and emit anonymized artifacts.

ASCII Diagram

